

Brownian dynamics vs Nosè-Hoover algorithm

Luciano Lanzotti

1 Introduzione

Il codice presentato simula l'interazione tra N particelle di massa $m = 1$ soggette ad un potenziale di Lennard-Jones con $\sigma = \epsilon = 1$ e raggio di cut-off $r_c = 2.5$. La simulazione è stata condotta nell'insieme canonico, utilizzando sia le equazioni di Nosè-Hoover, sia l'algoritmo BAOAB per la dinamica browniana. Le particelle si muovono in una scatola cubica di volume V ottenuto fissando la densità del sistema ρ^* . Si adottano condizioni periodiche al bordo. Infine, con queste scelte di σ, ϵ e m , il tempo caratteristico τ è pari ad 1, di conseguenza si scelgono passi di integrazione nell'intervallo $[0.001, 0.01]$.

2 Riassunto del codice

In questa sezione sono riassunti gli aspetti principali del programma, una descrizione più dettagliata è disponibile nei commenti. Per ottenere i file di dati è necessario creare nella cartella di lavoro una cartella chiamata **dati**. All'interno della quale si devono creare altre 4 cartelle chiamate:

- dt
- rdf
- ssd
- u

2.1 Classe particella

Nel file `particle.hpp` si definisce la classe `particella`, con i due metodi che implementano gli evolveri $e^{i\mathcal{L}_p\Delta t}$ e $e^{i\mathcal{L}_q\Delta t}$, sempre in questa classe si trovano i metodi per calcolare la forza e il potenziale tra due particelle, rispettivamente `fij()` e `vij()`. Inoltre si definisce la funzione `vijs()` per calcolare l'interazione riscalata tra due particelle, di seguito si riporta un estratto del codice:

```
1 ntype vijs(particleLJ P, pvector<ntype,3> L){
2     ntype ene;
3     pvector<ntype,3> Dr;
4     Dr = r - P.r;
5     Dr = Dr - L.mulcw(rint(Dr.divcw(L))); // Minimum image convention
6     ntype rn = Dr.norm(); // Calcolo della norma
7     ntype rsq = rn*rn; // Quadrato della norma
8     if (rsq < rc*rc) // Se la distanza e' minore del raggio di cut-off, allora interagiscono
9         ene = 4.0*epsilon*(pow(sigma/rn,12.0)-pow(sigma/rn,6)) - vcut; // Si osservi la
        sottrazione per vcut (il valore del potenziale al cut-off)
10    else
11        ene = 0;
12    return ene;
13 }
```

2.2 Classe parametri

Nel file `params.hpp` si definisce una classe per i parametri della simulazione, inoltre sono definiti dei metodi per modificare gli attributi in maniera più efficiente. Ad esempio `set_dt()` permette di cambiare il passo di integrazione insieme al numero di step per fare in modo che il tempo totale rimanga lo stesso. Il codice è riportato di seguito:

```

1 void set_dt(double new_dt){
2     dt = new_dt; // dt e' un attributo della classe
3     totsteps = (long int)rint(t_tot/new_dt); // Si ricalcolano i nuovi step totali, affinche'
        il tempo totale sia circa lo stesso.
4     if (eqsteps>0) // Se il numero di passi prima dell'equilibratura sono minori di 0, vuol
        dire che quest'ultima non e' introdotta nella simulazione. Dunque non si modifica
        eqsteps.
5     eqsteps = (long int)rint(totsteps*0.3);
6 }

```

2.3 Classe termostato Nosè-Hoover

Nel file `nh.hpp` è definita la classe `nh` per rappresentare il termostato di Nosè-Hoover. Sono presenti degli attributi per le variabili η , p_η e Q , quest'ultima viene calcolata come:

$$Q = 3NK_B T \tau^2 \quad \text{tale che } K_B = 1 \quad (1)$$

al momento della creazione di un oggetto di questo tipo. In aggiunta è presente il metodo per implementare l'evolvente $e^{i\mathcal{L}_\eta \Delta t}$ secondo:

$$\eta \rightarrow \eta + \frac{p_\eta}{Q} \Delta t \quad (2)$$

mentre $e^{i\mathcal{L}_{p_\eta} \Delta t}$ è rappresentato da:

$$p_\eta \rightarrow p_\eta + F \Delta t \quad \text{tale che } F = \sum_{i=1}^N m_i |\vec{v}_i|^2 - 3NT \quad (3)$$

infine l'evolvente $e^{i\mathcal{L}_{F_{NH}} \Delta t}$ è dato in questa forma:

$$\vec{v}_i \rightarrow \vec{v}_i \cdot e^{-p_\eta \Delta t / Q} \quad i = 1, \dots, N. \quad (4)$$

Si precisa che i metodi di questa classe sono pensati per evolvere tutte le particelle in una sola chiamata. In conclusione il metodo `energy()` calcola il contributo energetico del termostato dentro E_{NH} che sarebbe: $\frac{p_\eta^2}{2Q} + 3NT\eta$.

2.4 Classe dinamica browniana

Nel file `brownian.hpp` è presente la classe `brownian` per implementare l'evolvente $e^{i\mathcal{L}_G \Delta t}$ attraverso il metodo `brownian.expilG()`, la cui azione è mostrata di seguito:

$$\vec{v}_i \rightarrow \vec{v}_i \cdot e^{-\gamma \Delta t} + \frac{K}{m_i} \vec{G} \quad i = 1, \dots, N \quad (5)$$

dove $\gamma = 1$, $\vec{G}$ è una terna di numeri gaussiani, mentre K è dato da:

$$K^2 = T (1 - e^{-2\gamma \Delta t}) m_i \quad (6)$$

dove m_i è la massa della i -esima particella che stiamo evolvendo attraverso la relazione 5. Infine si precisa che `brownian.expilG()` è pensata per aggiornare una particella alla volta.

2.5 Classe simulazione

Nel file `sim.hpp` è costruita la classe `mdsim` per la simulazione in questione. Nel metodo `prepare_initial_conf()` si definisce il lato della scatola cubica facendo in modo che la densità sia quella indicata nella classe dei parametri, si pongono le particelle su un reticolo di tipo `simple cubic`, per ognuna si assegna una velocità iniziale v_i secondo una gaussiana con deviazione standard $\sqrt{T/m_i}$, poi si riscaldano tutte le velocità per fare in modo che la quantità di moto del centro di massa sia nulla.

Il metodo `calc_forces()` calcola la forza totale su una particella, che è pari alla somma delle forze che le altre $N - 1$ particelle applicano su di essa. Inoltre si utilizza questa funzione anche per calcolare il potenziale normale (U) e quello riscaldato (U_s).

Nel metodo `run()` si definisce il ciclo su tutti gli step temporali della simulazione, nel quale il passo con Nosè-Hoover è implementato in questo modo:

```

1 thermostat.expilpeta(pars.dt*0.5, parts); //thermostat e' un'istanza di nh
2 thermostat.expilFNH(pars.dt*0.5, parts);
3 thermostat.expileta(pars.dt*0.5);
4 velocity_verlet(); // Passo del velocity verlet, fa l'evoluzione di tutte le particelle
5 thermostat.expileta(pars.dt*0.5);
6 thermostat.expilFNH(pars.dt*0.5, parts);
7 thermostat.expilpeta(pars.dt*0.5, parts);

```

Se non viene eseguito il passo con Nosè-Hoover, allora si applica l'algoritmo BAOAB come segue:

```

1 for(int i=0; i<pars.Np; i++){
2     parts[i].expilp(pars.dt*0.5); // Evolutore degli impulsi
3     parts[i].expilq(pars.dt*0.5); // Evolutore delle posizioni
4     pbc(i); // Dopo aver cambiato le posizioni e' necessario applicare le pbc
5 }
6 calc_forces(Us, U); // Dopo aver aggiornato le posizioni le forze sulle singole particelle
   cambiano
7 for(int i=0; i<pars.Np; i++){
8     b.expilG(pars.dt, parts[i]); // Evolutore proprio della dinamica browniana
9     parts[i].expilq(pars.dt*0.5); // Evolutore delle posizioni
10    pbc(i); // Come prima si applicano le pbc
11 }
12 calc_forces(Us, U); // Ricalcoliamo le forze
13 for(int i=0; i<pars.Np; i++)
14     parts[i].expilp(pars.dt*0.5); // Aggiornamento degli impulsi

```

2.6 Classe per la radial distribution function

Nel file `radial.hpp` si implementa la classe `radial_distribution` utile a calcolare la radial distribution function $g(r_p)$, la cui formula è data da:

$$g(r_p) = \frac{n(r_p)}{N\rho 4\pi r_p^2 \Delta r} \quad (7)$$

dove $n(r_p)$ è il numero medio di particelle la cui distanza è compresa tra r_p e $r_p + \Delta r$. Quindi bisogna costruire un istogramma i cui bin sono incrementati di 2, se ad un dato step temporale si trova una coppia di particelle a distanza compresa in r_p e $r_p + \Delta r$. Di seguito si mostra un estratto del metodo `update_counts()` che esegue quanto appena descritto.

```

1 for (int i=0; i < Np; i++){
2     for (int j=i+1; j < Np; j++){
3         Dr = parts[i].r - parts[j].r; // Calcolo della distanza
4         Dr -= L.mulcw(rint(Dr.divcw(L))); // Applicazione delle PBC
5         rn = Dr.norm();
6         // Solo rn<r_max aggiorniamo l'elemento count[bin]
7         // Se rn>r_max la distanza e' fuori l'intervallo considerato
8         if (rn<r_max){
9             bin = (long int)(rint((rn/r_max)*n_bins));
10            counts[bin] += 2;
11        }
12    }
13 }
14 n_update += 1;

```

Il metodo `update_counts()` viene chiamato ad ogni step temporale nel ciclo dentro `mdsim.run()`. Alla fine della simulazione per far in modo che $n(r_p)$ rappresenti un valore medio bisogna dividere per il numero di volte che la funzione `update_counts()` è stata chiamata.

2.7 Classe per lo static structure factor

Per il calcolo dello static structure factor $S(k)$ si utilizza la classe `ssf` definita nel file `static.hpp`. Di seguito si riassumono brevemente alcuni presupposti teorici e si mostra in generale come funziona il codice. Per prima cosa, le condizioni periodiche sul bordo impongono che i soli vettori $\vec{k}$ permessi siano del tipo:

$$k_\alpha = \frac{2\pi}{L_\alpha} n_\alpha \quad \text{tale che } \alpha = x, y, z \quad (8)$$

dove $n_\alpha = 0, \pm 1, \pm 2, \dots$. Successivamente si ricorda la formula di $S(k)$:

$$S(k) = \frac{1}{N} \langle \rho(\vec{k}) \rho^*(\vec{k}) \rangle = \langle |\rho(\vec{k})|^2 \rangle \quad (9)$$

dove

$$\rho(\vec{k}) = \sum_{i=1}^N e^{i\vec{k} \cdot \vec{r}_i} \quad (10)$$

Allora per calcolare $S(k)$ ad ogni passo della simulazione vengono generate **total_points**= 1000 terne di interi n_x, n_y, n_z , dove ogni numero è scelto casualmente tra -15 e 15 , queste terne identificano dei vettori secondo la relazione 8. In questo caso il massimo modulo ottenibile è pari a $k_{\max} = 15\sqrt{3}\frac{2\pi}{L}$, assumendo che la scatola sia cubica ($L_x = L_y = L_z$). Per convenzione si definisce il binsize come $bs \equiv \frac{2\pi}{3L} = \frac{\Delta k}{3}$, ovvero la terza parte del modulo del vettore più "piccolo" ottenibile. Il motivo di questa scelta è che è possibile ottenere due vettori i cui moduli si differenziano per meno di Δk . Ad esempio $\vec{k}_1 = \Delta k(1, 1, 0)$ e $\vec{k}_2 = \Delta k(1, 1, 1)$, sono tali che $k_2 - k_1 \approx 1.73 - 1.41$.

Detto ciò si ottiene che il numero di bin necessari a coprire l'intervallo $(0, k_{\max})$ è la parte intera superiore del rapporto tra $k_{\max}$ e il binsize. In formule: **n_bins**=[$k_{\max}/bs$]. Successivamente si considerano due array con lunghezza **n_bins**: **count** e **rho2**. Gli elementi del primo contano quante volte è stato estratto un vettore $\vec{k}$ con un certo modulo, mentre quelli del secondo sono le somme di tutti i valori di $|\rho(\vec{k})|^2$ ottenuti per lo stesso modulo. La funzione che aggiorna questi due array si chiama **update_ssf()** ed è mostrata in parte di seguito:

```

1  std::complex<double> rho; // rho(k) definito come sum_j exp(i*kj*rj). Sarà un numero
    complesso.
2  pvector<double,3> n; // Vettore che conterra' la terna di n, per convenienza creiamo di tipo
    double.
3  pvector<double,3> k_vec; // pvector che conterra' il vettore k generato casualmente.
4  double k; // Modulo del vettore casuale.
5  long int bin; // Indice del bin a cui k appartiene
6  for(int p=0; p<total_points; p++){ // For sui 2000 vettori casuali
    // Generiamo una terna casuale di numeri interi nx,ny,nz.
7      for(int i=0; i<3; i++){
8          n(i) = (double)random_int(-n_max,n_max);
9      }
10     k_vec = n.divcw(L)*2*M_PI; // Dalla terna di n otteniamo un vettore
11     k = k_vec.norm(); // Ne calcoliamo la norma
12     bin = (long int)rint(k/bin_size); // Calcoliamo a quale bin corrisponde quella norma
13     rho = evaluate_rho(k_vec, parts, Np); // Calcoliamo rho associato a quel vettore
14     rho2[bin] += std::norm(rho); // Aggiorniamo rho2 nella posizione bin-esima
15     count[bin] += 1; // Aggiorniamo count nella posizione bin-esima
16 }

```

In conclusione finita la simulazione si calcola $S(k) = \frac{1}{N} \langle |\rho(\vec{k})|^2 \rangle$ come segue:

```

1  for(int bin=0; bin<n_bins; bin++){
2      // Se il bin e' vuoto sono possibili due casi:
3      // 1. Poiche' i vettori k sono discreti, allora non sono possibili vettori con modulo nel
        range del bin.
4      // 2. Per caso non sono stati estratti vettori con modulo in quel range.
5      // Ponendo S_k[bin] a 0 se count[bin] e' nullo, evitiamo una divisione tra due 0.
6      if (count[bin]==0){
7          S_k[bin]=0.0;
8      }
9      // rho2[bin]/count[bin] = <rho2[bin]>
10     // Allora dividendo anche per il numero di particelle, si ottiene S_k
11     else{
12         S_k[bin] = rho2[bin]/(Np*(double)count[bin]);
13     }
14 }
15 }

```

dove **Np** rappresenta il numero di particelle N . Il costrutto decisionale dentro il ciclo **for** è utilizzato per evitare una divisione tra zeri. Inoltre un valore nullo di $S(k)$ non viene preso in considerazione per costruire i grafici dello static structure factor in figura 4.

3 Risultati della simulazione

3.1 Fluttuazioni dell'energia nell'algoritmo Nosè-Hoover

Nell'algoritmo di Nosè-Hoover l'energia conservata è data da:

$$E_{NH} = \sum_{i=1}^N \frac{|\vec{p}_i|^2}{2m} + V(\vec{q}_1, \dots, \vec{q}_N) + \frac{P_\eta^2}{2Q} + 3N K_B T \eta. \quad (11)$$

Definiamo una misura della fluttuazione dell'energia nel seguente modo:

$$\sigma(E) = \sqrt{\sum_{i=1}^k \frac{1}{k} |E_{NH}(0) - E_{NH}(i \cdot \Delta t)|^2} \quad (12)$$

dove $i = 0$ è il primo step dopo aver raggiunto l'equilibratura. Ci aspettiamo che $\sigma(E)$ sia quadratica con il passo di integrazione Δt . Per mostrare ciò, sono stati scelti 5 valori di Δt compresi tra 0.001 e 0.01. Per ognuno di essi è stata lanciata una simulazione, con 216 particelle, 6 per lato nella configurazione iniziale, densità $\rho^* = 0.4$, temperatura $T = 2$ e tempo totale $t_{tot} = 3$. Inoltre si è considerata raggiunta l'equilibratura dopo che nella simulazione era passato un tempo maggiore di quello caratteristico τ .

In figura 1 si mostra il grafico dell'energia E_{NH} in funzione del tempo per le 5 simulazioni, si osserva che al diminuire del passo di integrazione le energie fluttuano intorno a valori sempre più alti, ciò suggerisce che $E_{NH}(t)$ stia convergendo al valore dettato da un'hamiltonina shadow. Invece nell'immagine 2 sono mostrate le fluttuazioni $\sigma(E)$ in funzione del passo di integrazione Δt . Il coefficiente angolare della retta nel grafico di destra è 1.94, prossimo al valore esatto di 2.

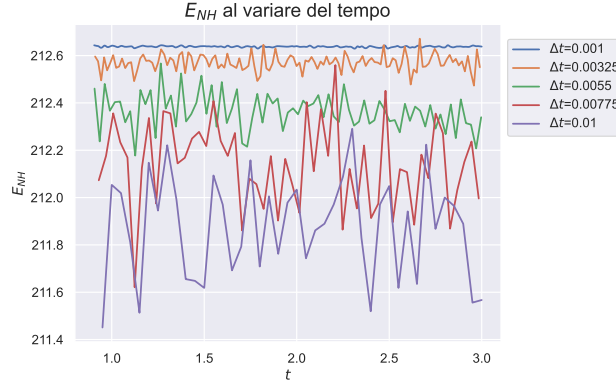


Figura 1: Energia conservata E_{NH} in funzione del tempo, si osserva che sono stati raccolti dati solo per tempi maggiori del tempo caratteristico del sistema, in questo modo si è sicuro che l'equilibratura sia avvenuta.

3.2 Radial distribution function

Per valori di $\rho^* = 0.1, 0.2, 0.3, 0.4$ si esegue una simulazione con 216 particelle, 6 per lato nella configurazione iniziale, temperatura $T = 1.4$, passo di integrazione $\Delta t = 0.01$, tempo totale $t_{tot} = 3$. Per avviare in automatico queste quattro simulazioni si usò il codice dentro al file `test_g.cpp`. La chiamata del metodo `radial_distribution.update_counts()` dentro il ciclo `for` del metodo `mdsim.run()` è avvenuta solo dopo che nella simulazione è passato un tempo pari a quello caratteristico, in modo tale da essere sicuri di aver raggiunto l'equilibratura. Si mostrano in figura 3 le radial distribution function ottenute per i diversi valori di densità nel caso di dinamica browniana e di algoritmo Nosè-Hoover. Come aspettato la $g(r)$ ha un picco nel minimo del potenziale di Lennard-Jones, per poi diminuire ed oscillare intorno ad 1. Successivamente per distanze maggiori di $L/2$, cioè metà del lato della scatola cubica, la $g(r)$ tende a 0. Ciò si spiega, poiché, per effetto del volume finito, diventa sempre più improbabile trovare particelle a grandi distanze.

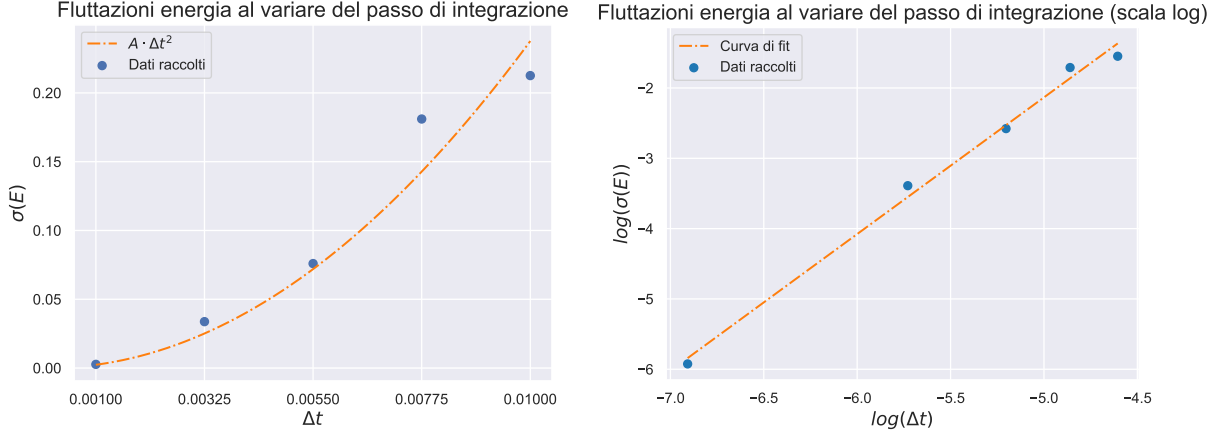


Figura 2: A sinistra l'andamento della fluttuazione dell'energia confrontato con il suo comportamento atteso parabolico. A destra lo stesso grafico ma in scala logaritmica, si osserva che i punti si dispongono su una retta con coefficiente angolare 1.94.

3.3 Static structure factor

Per valori di $\rho^* = 0.1, 0.2, 0.3, 0.4$ si esegue una simulazione con 125 particelle, 5 per lato nella configurazione iniziale, temperatura $T = 1.4$, passo di integrazione $\Delta t = 0.01$, tempo totale $t_{tot} = 3$. In questo caso si sceglie un numero minore di particelle, perché si osserva che un N maggiore non produce delle curve più regolari nella figura 4. Si lanciano le simulazioni in automatico dal file `test_ssf.cpp`. La funzione `update_ssf()` che esegue l'aggiornamento di `count` e `rho2`, viene chiamata dentro ciclo `for` di `mdsim.run()`, solo una volta passato un tempo pari a quello caratteristico del sistema. In questo modo si è sicuri di aver raggiunto l'equilibratura. I risultati sono mostrati in figura 4. Si osserva che al diminuire della densità, $S(k)$ tende ad una funzione costante uguale ad 1. Ciò si spiega ricordando una formula alternativa per il fattore di struttura:

$$S(\vec{k}) = 1 + \rho \int d\vec{r} e^{-i\vec{k} \cdot \vec{r}} g(\vec{r}). \quad (13)$$

3.4 Energia interna al variare del passo di integrazione

Si lancia una simulazione per diversi valori del passo di integrazione $\Delta t = \{0.005, 0.01125, 0.0175, 0.02375, 0.03\}$, sia nel caso di Nosè-Hoover che per la dinamica browniana. Si utilizzano 125 particelle, 5 per lato nella configurazione iniziale, una densità $\rho^* = 0.4$, una temperatura di $T = 1.4$ e un tempo totale $t_{tot} = 3$. Per lanciare in automatico queste simulazioni si utilizza il file `test_U.cpp`. Successivamente, definita l'energia interna in questo modo:

$$U = \sum_{i=1}^N \frac{|\vec{p}_i|^2}{2m} + V(\vec{q}_1, \dots, \vec{q}_N) \quad (14)$$

se ne calcola il valore medio su ognuna delle simulazioni. Sono stati considerati solo i valori di energia calcolati dopo che il tempo della simulazione ha superato τ . In figura 5 sono mostrati i risultati. In conclusione si osserva che nell'algoritmo di Nosè-Hoover l'energia interna media cresce al variare del passo di integrazione, diversamente la dinamica browniana sembra non dipendere da Δt .

4 Extra: studio dello static structure factor per densità maggiori

Come illustrato nella sezione 3.3, vengono eseguite quattro simulazioni per calcolare $S(k)$. I parametri rimangono invariati, ma le densità sono ora $\rho^* = \{0.5, 0.6, 0.7, 0.8\}$. In figura 6 sono mostrati i risultati. In questo caso le oscillazioni del fattore di struttura sono molto più evidenti, conseguenza che l'integrale nel termine 13 pesa maggiormente. Inoltre si verifica meglio che $S(k)$ tende a 0 per k piccoli.

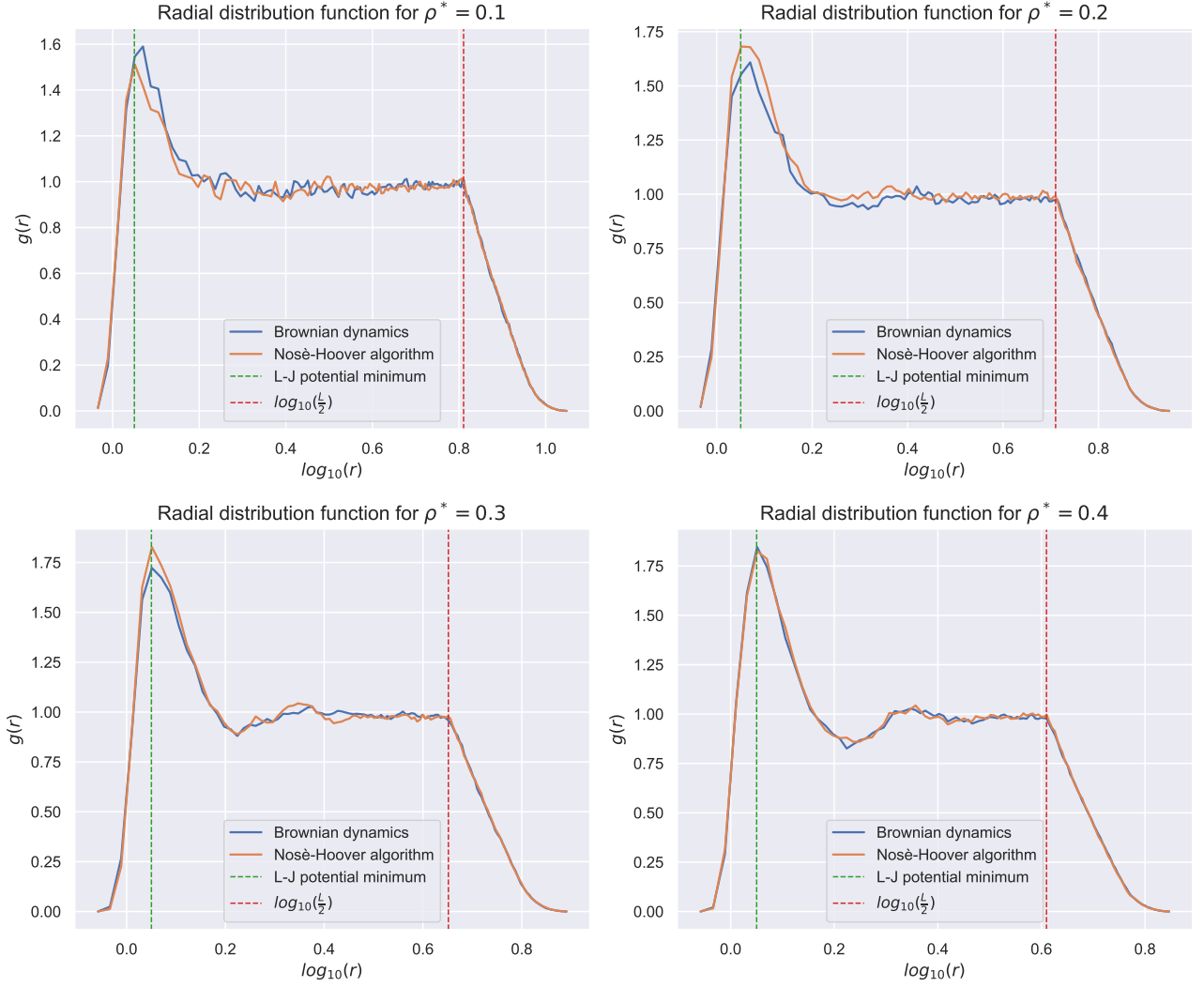


Figura 3: Radial distribution function per diversi valori della densità nel caso di dinamica browniana e algoritmo di Nosè-Hoover.

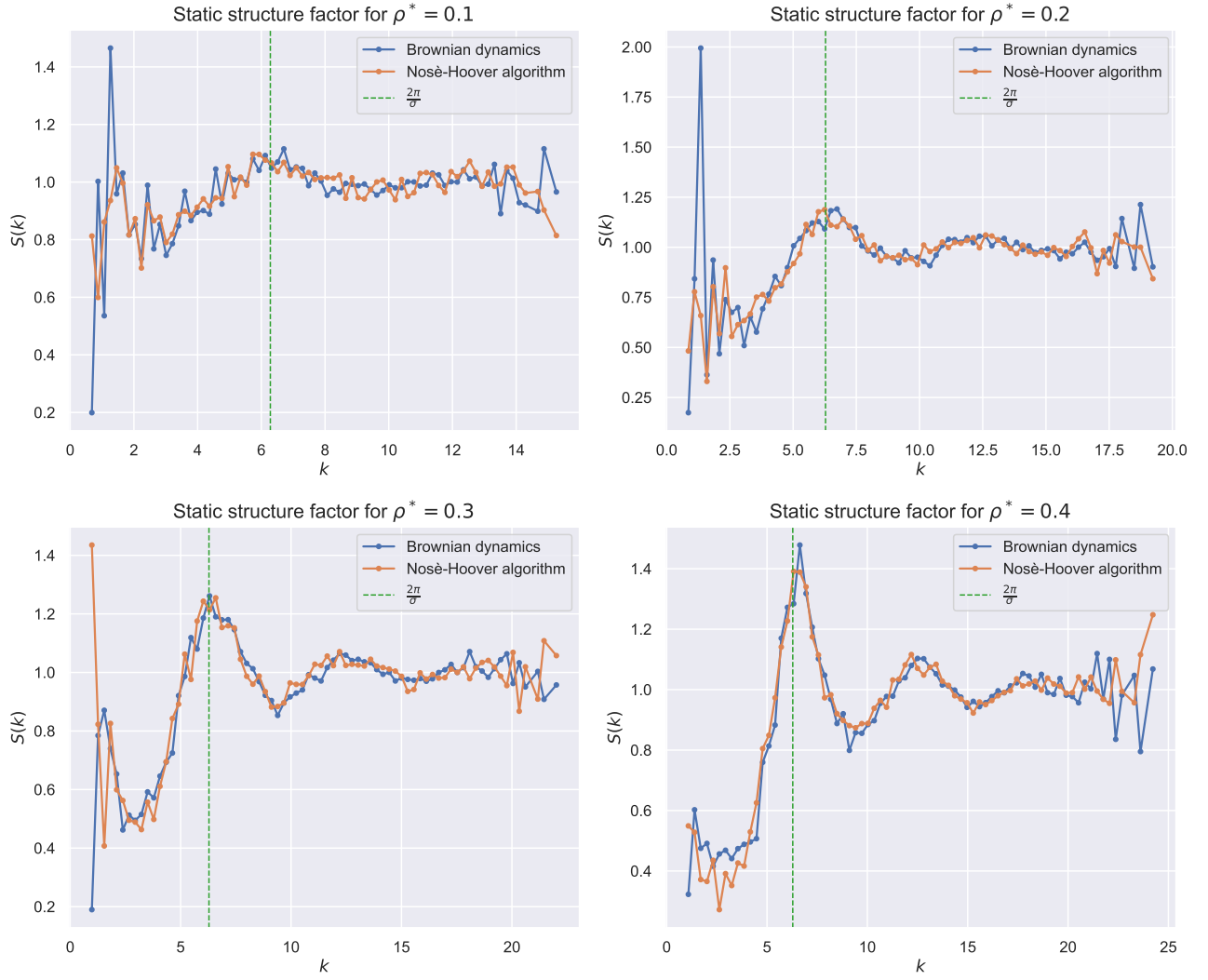


Figura 4: Static structure factor per diversi valori di ρ^* . Il picco di $S(k)$ si trova in corrispondenza di $\frac{2\pi}{\sigma}$, dove σ è il diametro della sfera. Inoltre non sono mostrati valori di k che tendono a 0^+ , poiché il modulo del vettore più 'piccolo' ottenibile secondo la 8 è $\frac{2\pi}{L}$.

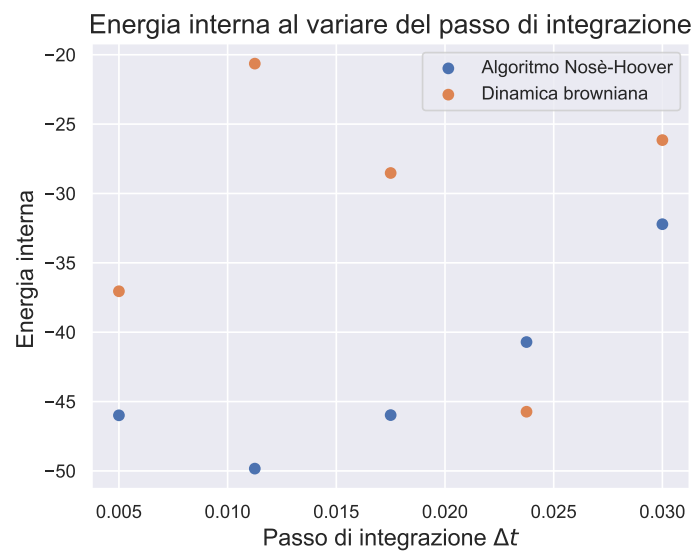


Figura 5: Andamento dell'energia interna al variare del passo di integrazione nel caso di dinamica browniana e algoritmo di Nosé-Hoover.

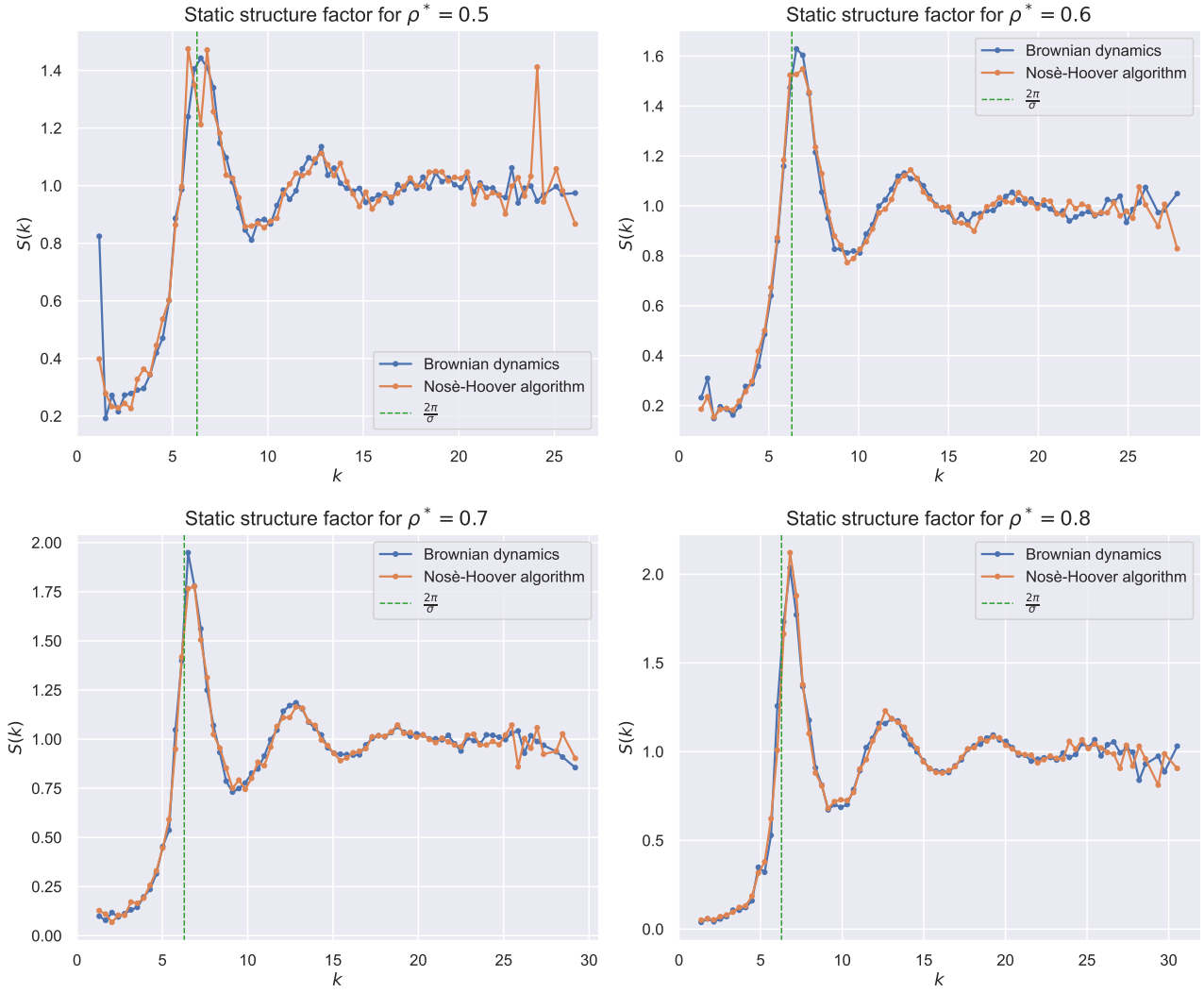


Figura 6: Static structure factor per densità del sistema più grandi.